KOD GERÇEKLİĞİ DENETİMİ · BULGU LİSTESİ

Kaynaklar ile belgeler arasındaki uyumsuzluklar

Beş ciltli kitap setleri, yüzey matrisleri, CLI yardım metni ve kütüphane atlasları; `src/`, `libsx/`, `runtime_ext/`, `manifests/` ve `bin/` içeriğiyle satır satır karşılaştırıldı. Aşağıdaki 25 bulgu, dosya ve satır kanıtına dayanır.

10	4	6	5
Kitap ↔ kaynak çelişkisi	CLI/mimari belgesi hatası	Kaynak içi tutarsızlık	Kütüphane/manifest boşluğu

A · Kitaplar kaynağa uymuyor

Bu grup en kritik olan; çünkü kitaptaki örnekleri yazan okuyucu derleyici hatası alır.

A1 · SQRT kitapta çalışır gösteriliyor, parser reddediyor

En yaygın hata bu. `parser_shared.fbs` satır 448–449 açıkça şunu yapıyor:

```
Case "SQRT"
    ps.lastError = "SQRT removed; use SQR"
    Return 0
```

Buna karşılık kitaplarda çalışan örnek olarak geçiyor:

- `pck.md` / `pckg.md` s.508 fonksiyon tablosu: “SQR / SQRT | Karekök. | k = SQRT(16)”
- `pck.md` s.531 ve s.803: `PRINT "Karekoku="; SQRT(sayi)` ve z-skoru formülü
- `pck2n.md` s.2228, `pck2g.md` s.1458: `PRINT SQRT(10)`
- `pck2d.md` s.155, 1713, 1900–1905, 2112: SQRT ayrı bir fonksiyon maddesi olarak, sözdizimi ve örnekle anlatılıyor

Durumu ağırlaştıran nokta: `lexer_keyword_table.fbs` s.44 `"sqrt"`’ı hâlâ anahtar sözcük sayıyor. Yani kullanıcı “tanınmayan fonksiyon” değil, dilin içinden gelen bir ret mesajı alıyor. **Öneri:** ya kitaplardaki tüm SQRT geçişleri SQR’a çevrilsin, ya da parser SQRT’ı SQR’a takma ad olarak kabul etsin. İkisinden biri seçilmeli — şu an ikisi de yapılmamış.

A2 · `INKEY()` argümansız kullanılamıyor

`pck.md` s.508 tablosu “INKEY GETKEY | Klavye okuma. | k = INKEY()” diyor. Kaynakta ise `parser_shared.fbs` s.457 `INKEY` için **1–2 argüman** zorunlu kılıyor; sıfır argümanlı biçim yalnız `GETKEY()` için geçerli (s.459, tam 0 argüman). Tablodaki örnek satırı olduğu gibi yazılırsa derlenmez.

A3 · `SIZEOF(x)` örneği geçersiz

`pck.md` s.508: "SIZEOF OFFSETOF | Boyut ve alan kayması. | n = SIZEOF(x)". Ancak `ValidateSizeOfCall` (s.340–350) yalnız **tip adı** veya **dizi tanımlayıcısı** kabul ediyor; sıradan bir skaler değişken için "SIZEOF expects a type name or array identifier" hatası veriyor. Kitabın örneği `SIZEOF(I32)` gibi yazılmalı.

A4 · `EOF(#1)` — `#` ifade içinde geçersiz

`pck.md` s.508 "bitti = EOF(#1)" yazıyor. Projenin kendi kanonik kararı (`README.md`) ise net: *"`#` yalnız dosya kanalı bağlamında kullanılır: `PRINT #1, INPUT #1`."* İfade içindeki `EOF(#1)` bu kurala aykırı; doğru yazım `EOF(1)`.

A5 · `PARALLEL` kitapta ve yüzey matrislerinde yaşamaya devam ediyor

Kaynakta tek doğru yazım `PARALEL` (`lexer_keyword_table.fbs` s.18). Projenin kendi yasak listeleri de bunu tekrarlıyor (ADIM37/38/40/42: "PARALLEL yok, doğru anahtar PARALEL"). Buna rağmen şu belgeler `PARALLEL`'i gerçek bir anahtar gibi listeliyor:

- `pck2n.md` s.26, madde 18 — "Dil tarafından tanınan sözcük veya yerel ad"
- `docs/matrix/UXB_LEXER_KEYWORD_ENVANTERI.md` s.143 — kaynağı "lexer" olarak gösteriyor
- `docs/matrix/UXB_KEYWORD_LAYER_MATRIX.md` — UXB-SURFACE-031, "ADD_PARSER_HANDLER" önerisiyle
- `docs/matrix/UXB_268_YUZEY_ANAHTAR_ENVANTERI.md` s.190

Bu üç matris dosyası bayat. Daha kötüsü, ikisi "parser handler ekle" diye **yanlış yönde iş öneriyor**; oysa proje kararı tam tersi.

A6 · `^` kitapta "bit işlemi", kaynakta `XOR`

`pck.md` s.456 işlemci tablosunda 7. satır `& | ^ ~` için "Bit düzeyi işlemler" diyor ve ^'ın ne yaptığını hiç açıklamıyor. Kaynak nettir — `parser_adim08_operator_contract.fbs` s.20 `Case "^": Return "XOR"`: yani `^` mantıksal/bitsel XOR'a kanonikleşiyor (öncelik 70) ve **üs alma değildir**. Üs alma `**` (öncelik 140). BASIC geleneğinden gelen okuyucu için bu sessiz bir tuzak; kitapta açıkça yazılmalı.

A7 · `|>` gerçek bir işleç, kitapta "ayrılmış işaret"

`pck.md` s.456 satır 16, `@ # ? ! |>` için "Lexer yüzeyinde ayrılmış özel işaretler" diyor. Oysa `ParserOpPrecedence` s.56–57 `|>`'a gerçek ikili işleç önceliği (30, mantıksal işleçlerin altında, üçlü `?:`'nin üstünde) veriyor. Dilin boru hattı işleci belgelenmemiş durumda.

A8 · Logic16 ailesi giriş cildinde yok

`pck.md` s.456 satır 6 mantıksal işleç olarak yalnız `AND OR NOT XOR` sayıyor. Kaynakta ise `ParserOpIsLogic16` (s.85–93) şu 11 işleci ayrı bir sınıf olarak tanıyor:

```
NAND  NOR  XNOR  IMP  CIMP  NIMP  NCIMP  IDA  IDB  NOTA  NOTB
```

Ayrıca `NXOR / EQV / EQA / IFF` hepsi `XNOR`'a takma ad; `&& → ANDALSO`, `|| → ORELSE`, `== → =`, `!= → <>`, `ZERO → FALSE`, `ONE → TRUE`. Bunların hiçbiri giriş cildinin işlemci tablosunda yok — dilin en özgün yüzeyi belgesiz kalmış. (Aynı kitabın ileri bölümündeki anahtar listesi bunları sayıyor; yani kitap kendi içinde de tutarsız.)

A9 · Bileşik atama işleçlerinin yarısı belgesiz

`pck.md` satır 2 yalnız `+= -= *= /= \= %=` sayıyor. Kaynak (aynı sözleşme dosyası, s.60 ve s.70) bunlara ek olarak `&= |= ^= <<= >>= **= ROL= ROR= =+ =-` tanıyor. Özellikle `=+` ve `=-` hiçbir kitapta geçmiyor.

A10 · `STRUCT / UNION / ABI / PACK / ALIGN` veri yapısı tablolarında yok

Kaynakta bunlar tam bir C-ABI kayıt yüzeyi (`parser_stmt_decl_core.fbs` s.1058–1147):

```
STRUCT Ad [ABI C] [PACK n] [ALIGN n] ... END STRUCT
UNION  Ad [ABI C] [PACK n] [ALIGN n] ... END UNION
' n ∈ {1,2,4,8,16,32,64}
' TYPE → ABI UX zorunlu; STRUCT/UNION → ABI C zorunlu
' TYPE üzerinde PACK/ALIGN yasak: "TYPE: PACK/ALIGN are reserved for STRUCT/UNION ABI C"
```

Dil yüzeyi analizi belgesi de dahil, incelenen tablolarda yalnız `TYPE` anlatılıyor; `STRUCT/UNION` ve `ABI/PACK/ALIGN` seçenekleri geçmiyor. Aynı şekilde `IsBuiltinTypeKeyword` (s.124) içindeki `BIGI / BIGINT / EXTENDED / EXT` ve `CanonicalTypeNameForParser` (s.131–160) içindeki `BIGFLOAT / BIGDECIMAL / BIGINTEGER / FLOAT128 / QUAD / DOUBLE / DBL / DBLE / SINGLE / SNG / SNGLE / INTEGER / INT` takma adları belgelenmemiş. Kullanıcı `DIM x AS DOUBLE` yazabildiğini bilmiyor.

B · CLI ve mimari belgeleri

Karşılaştırma tabanı: `bin/uxb_yardim.txt` ve `docs/UXB_MIMARI_NOTU_BACKENDS_CLI_JSON_I18N.md` ↔ `src/cli/cli_registry_adim41.fbs` ve `cli_verb_dispatch.fbs`.

B1 · `--enable-mir-x64-experimental` diye bir seçenek yok

Mimari notu CLI hedefini tarif ederken bu adı üç yerde kullanıyor (`--emit-x64-nasm --codegen-source MIR --enable-mir-x64-experimental` gibi). Kayıtta ise seçenek `--enable-mir-x64` (items 100) ve dosyanın kendi örnek satırı tercih edilen biçimi `--x64-mode=MIR` olarak veriyor; `--enable-mir-x64` ona takma ad. Belgedeki komut satırları olduğu gibi kopyalanırsa çalışmaz.

B2 · x86 verb'i belgesiz

Yardım metni 19 verb sayıyor. `cli_verb_dispatch.fbs` s.17 ise 20 tanesini kabul ediyor — listede `x86` da var, yardımda yok.

B3 · Kayıtlı seçeneklerin üçte biri belgesiz

Yardım metni ~80 seçenek anlatıyor; kayıt **171** tanesini tanımlıyor. Belgesiz kalanlar arasında günlük kullanım açısından önemli olanlar:

```
--pp / --preprocessor        preprocessor A|B seçimi
--x64-mode / --codegen-source  AST|MIR backend seçimi
--source / -s                kaynak dosya açık biçimi
--message-lang / --language  tanı dili (i18n sözleşmesinin asıl anahtarı)
--error-language-file        harici hata dili dosyası
--error-log-out / --error-summary-json-out / --error-to-terminal
--preprocessed-out / --preprocess-result-json-out
--manifest-out / --run-manifest-json-out / --browser-index-out
--canonical-mir-json-out / --semantic-mir-json-out
--semantic-enums/-layouts/-types-json-out
--ast-exec-result / --ast-live / --ast-stdout / --ast-stderr-out
--mir-exec-result / --mir-live / --mir-stdout / --mir-stderr-out
--interpreter-compare-json-out   AST⇔MIR karşılaştırma
--vscode-diagnostics-json-out    IDE entegrasyonunun beslendiği çıktı
--type-cast-report-json-out / --type-infer-report-csv-out
--parser-surface-contract-json-out / --all-diagnostics-json-out
--active-bind-plan-json-out / --runtime-route-matrix-json-out
--x64-ast-report / --x64-mir-report-json-out
--mir-x64-required-opcode-gate-json-out
--validate-all-report-json-out / --backend-matrix-json-out
--wasm-report-json-out / --wasm-wat-out / --js-runtime-out / --js-report-json-out
--extfp-diagnostics / --original-source-out / --semantik / --x64
--target-x86 / --target-js / --target-browser / --target-wat / --target-wasm
```

Yardım metninin kendi iddiası “derleyicinin iç organlarını göstermek” olduğuna göre, bu boşluk doğrudan ürünün ana vaadini zayıflatıyor. Ayrıca yardım `--target <hedef>`’in `x64, js, wat, wasm, browser, x86` aldığını söylüyor ama kayıtta bunların her biri ayrıca bağımsız `--target-*` bayrağı olarak da duruyor; hangisinin kanonik olduğu belirsiz.

B4 · i18n: sözleşme var, belge yok

Mimari notu “Bu adımda i18n sadece sözleşme olarak yazıldı” diyor ve `uxb/i18n/tr.json`, `en.json` yolunu veriyor. Gerçekte hem `i18n/` klasörü hem `src/error_i18n_template.csv` ve `error_i18n_seed_from_logs.csv` mevcut, CLI’de `--message-lang`, `--language`, `--error-language-file` tanımlı — yani iş yapılmış, belge güncellenmemiş. Format da farklı: not JSON diyor, kaynak CSV tohum dosyası kullanıyor.

C · Kaynak içi tutarsızlıklar

Bunlar belge hatası değil; kodun kendi katmanları arasındaki çelişkiler. Önceki denetim raporunun (§10) kapsamadığı yeni bulgular.

C1 · "=" öncelik tablosunda iki kez geçiyor — ikinci dal ölü kod

`parser_adim08_operator_contract.fbs` içinde `ParserOpPrecedence`:

```
Case "=", "<>", "<", "<=", ">", ">="        ' s.42
    Return 90
...
Case "=", "+=", "-=", ... , "=+", "=-"      ' s.60
    Return 10
```

FreeBASIC `Select Case` ilk eşleşmede durur. Yani `"="` **her zaman** 90 döner; atama önceliği (10) yalnız bileşik işleçlere uygulanır. Dosyanın kendi başlık yorumu *"= is context-sensitive"* diyor — ama bu fonksiyon bağlamı hiç göremiyor, dolayısıyla iddia tek bir öncelik fonksiyonuyla karşılanamaz. Ya ikinci `"="` kaldırılıp niyet yorumla açıklanmalı, ya da fonksiyona bağlam parametresi eklenmeli.

C2 · İşleç yüklemlerinin yarısı kanonikleştirme yapmıyor

Aynı dosyada `ParserOpPrecedence` ve `ParserOpIsLogic16` girdiyi `ParserOpCanonical()`'dan geçiriyor; `ParserOpIsRightAssociative` (s.66) ve `ParserOpIsAssignment` (s.76) ise ham `UCase(Trim())` kullanıyor. Sonuç: `"=="`, `"&&"`, `"^"`, `"NXOR"`, `"ZERO"` gibi takma adlar hangi yüklemin sorduğuna göre farklı sınıflanıyor. `NXOR` öncelik fonksiyonunda XNOR (45) sayılırken sağ-birleşimlilik sorgusunda tanınmıyor. Şimdilik zararsız görünüyor ama sözleşme dosyasının tek bir normalizasyon kuralı olmalı.

C3 · `PACK` / `ALIGN` yalnız ayrıştırılıyor, hiçbir katman tüketmiyor

Parser, kayıt bildiriminden şu sözleşme dizesini üretiyor (`parser_stmt_decl_core.fbs` s.1147):

```
recordContract = recordKind & ";ABI=" & abiKind & ";PACK=" & ... & ";ALIGN=" & ...
```

`src/layout`, `src/semantic/layout`, `src/type_binding`, `src/mir` ve `src/codegen/x64` klasörlerinde `recordContract`, `PACK` veya `UNION` geçen tek satır yok. Dahası `layout_engine.fbs` (52 satır) her kullanıcı kaydını şöyle işliyor:

```
Case "TYPE_STMT", "CLASS_STMT", "INTERFACE_STMT"
    UXBStep3LayoutRecordAdd ..., sz, UXBStep3TypeAlignBytes("TYPE_PTR"), "USER"
```

Üç sonuç: **(a)** bildirilen `ALIGN n` yok sayılıyor, her kayıt işaretçi hizasıyla kaydediliyor; **(b)** `PACK n` yok sayılıyor; **(c)** STRUCT/UNION bilinçli olarak `TYPE_STMT`'i yeniden kullandığı için (s.70 yorumu) layout katmanı **UNION'ı STRUCT'tan ayırt edemiyor** — union boyutu alanların en büyüğü değil, toplamı olarak çıkıyor. C ABI birlikte çalışabilirlik iddiası bu haliyle doğrulanmış sayılamaz. `SIZEOF`/`OFFSETOF` ve `--layout-report-json-out` çıktıları da bu yüzden şüpheli.

C4 · Preprocessor: "kanonik" ama A kümesinde ölümcül olan 25 direktif

`preprocess_parser.fbs` içindeki `UXBPreParseIsCanonicalCommand`, PreproesssorB'ın web/manifest komutlarını (`BUNDLE_NAME, BUNDLE_VERSION, BUNDLE_ID, AUTHOR, LICENSE, BUILD_ID, SANDBOX_ROOT, BROWSER_ENTRY, SCRIPT_ENTRY, STYLE_ENTRY, DATA_JSON, CSP, SECURITY_NOTE, CAPABILITY, EXPORT_BASIC, API_ENDPOINT, ROUTE, REQUIRE_ASSET, HOST_IMPORT, GEN_FILE, FEATURE, TEST_EXPECT, DOC`) **koşulsuz** kanonik sayıyor. `preprocess_engine.fbs` s.2023 ise bunları yalnız B kümesi aktifken yönlendiriyor:

```
If UXBPreprocessorIsBActive() <> 0 Then
    sc-camel-b-status = UXBPreHandleWebManifestDirective(...)
...
If UXBPreParseIsCanonicalCommand(head) = 0 Then ... META_UNKNOWN_DIRECTIVE
```

Varsayılan küme A (`preprocess_selector.fbs`: boşsa "A"). Dolayısıyla `%%PP B` yazmadan `%%ROUTE` veya `%%BUNDLE_NAME` kullanan bir dosya kanonik kapıyı geçiyor, `Select Case`'de eşleşme bulamıyor ve `Case Else`'de `META_UNKNOWN_DIRECTIVE` ile **sert hata** alıyor. Mesaj yanıltıcı: direktif bilinmiyor değil, yanlış kümede. Hata metni "bu komut %%PP B gerektirir" demeli.

C5 · Üç direktif hiçbir kümede işlenmiyor

`EMBEDBASE64`, `GEN_BLOCK` ve `GEN_END` kanonik listede var; ne A motorunda (`preprocess_engine.fbs` — `EMBEDTEXT` var, base64 karşılığı yok) ne PreproesssorB'da (`preprocess_web_manifest_commands.fbs`) karşılık dalları yok. Her iki kümede de kesin hata verirler. Ya uygulanmalı ya kanonik listeden çıkarılmalı.

C6 · Preprocessor komut yüzeyinin tamamı belgesiz

Kullanıcının sorduğu "preprocessor komutları" gerçekte **~120 %% direktifi** — koşullu derleme (`IF/IFN/IFC/ELIF/ELSE/ENDIF`), makro (`DEFINE/MACRO/UNDEF`), meta döngü (`FOR/FOREACH/REPEAT/BREAK/CONTINUE`), varlık boru hattı (`ASSET_*/COPY_DIR/HASHFILE/FINGERPRINT_ASSETS/MINIFY`), gömülü blok üreticileri (`JS_BEGIN/WAT_BEGIN/HTML_BEGIN/CSS_BEGIN/JSON_BEGIN/MD_BEGIN`), güvenlik kapıları (`SAFE_PATHS/ALLOW_RAW_JS/ALLOW_RAW_WAT/REQUIRE_HOST_BINDING/ASSERT_META`) ve iki ayrı komut kümesi (A/B, `%%PP` ile seçiliyor). İncelenen kitap ve kılavuzlarda bu yüzeyin sistematik bir tablosu yok; en fazla `INCLUDE` ve `IMPORT` ifade düzeyinde anlatılıyor. Bu, tek başına ayrı bir cilt hak eden en büyük belge boşluğu.

D · Kütüphane kaynağı ↔ kütüphane belgeleri

D1 · Fonksiyon atlası gerçek yüzeyin %40'ını gösteriyor

Örnek olarak `uxmath` alındı. Üç katman:

Katman	Dosya	Fonksiyon
C çekirdek	`runtime_ext/uxmath/uxmath.h`	42
uXBasic sarmalayıcı	`libsx/uxmath/uxmath.bas`	42
Belgelenen	`docs/UXMATH_FUNCTION_ATLAS.csv`	**17**

Atlasta hiç geçmeyen 25 sarmalayıcı: `Version, VecFree, VecSet, VecGet, VecCount, VecClear, VecDataPtr, PolyDerivativeCoeff, PolyIntegralCoeff, NumericDerivativePoly, ODE_Euler_Logistic, DSigmoidFromOutput, DReLU, Tanh, DTanhFromOutput, MatFree, MatRows, MatCols, MatSet, MatGet, MatFill, MatRandomUniform, MatAdd, MatSub, MatTranspose`. Yani **vektör okuma/yazma, matris okuma/yazma ve tüm bellek serbest bırakma çağrıları belgesiz** — atlasla çalışan biri `MatCreate` yapıp `MatSet`'in varlığını bilemez, matrise değer bile yazamaz.

Aynı desen diğer atlaslarda da beklenir; 18 modül için `*_FUNCTION_ATLAS.csv` dosyaları `libsx/` sarmalayıcılarından otomatik üretilmeli, elle yazılmamalı.

D2 · Atlasta yazım farkı

Atlas `UXMATH_Relu` ve `UXMATH_MatApplyRelu` yazıyor; sarmalayıcıdaki gerçek adlar `UXMATH_ReLU` ve `UXMATH_MatApplyReLU`. Ayrıca atlas tutamaç tipini `U64` gösteriyor, C tarafı `void*` — bu ikincisi `README_TR.md`'deki "45 opaque handle imzası PTR yerine U64 yapıldı" kararıyla bilinçli, sorun değil; belgelenmesi yeterli.

D3 · Manifest 46 DLL bildiriyor, 5'i `bin/`'de yok

`manifests/uxb_library_system.json` ile `bin/` karşılaştırıldığında eksik olanlar:

DLL	Manifest katmanı	Ek durum
`uxraylib.dll`	multimedia-primary	`libraylib.dll`, `glfw3.dll` bağımlılıkları da yok
`uxsdl3.dll`	multimedia-fallback	`runtime_ext/uxsdl3/` klasörü de yok
`uxterminal.dll`	—	kaynak var, çıktı yok
`uxlogic.dll`	—	Lua/Prolog yolu; kaynak var, çıktı yok
`uxclz.dll`	—	kaynak var, çıktı yok

Bu, `UXB_SPRITE_OYUN_TASARIM_REHBERI` ve `UXB_SPRITE_TERMINAL_LUA_PROLOG_FUZZY` belgelerini doğrudan etkiliyor: anlattıkları oyun/terminal/Lua yüzeyi bu dağıtımda çalışmaz. `examples/games/sprite_collision_game.uxb` örneği de aynı sebeple koşulamaz.

> **D4 · `bin/`'de manifestte olmayan DLL'ler**
>
> `uxcabi.dll` (C ABI köprüsü, `UXB_STRUCT_UNION_UXCABI` yamasının ürünü) kütüphane sisteminde bildirilmemiş; ayrıca `uxb_abi10_test.dll`, `uxb_ffi_contract_test.dll`, `uxcabi_aggregate_test.dll`, `uxonnx_test.dll` test artefaktları dağıtım klasöründe duruyor. Manifest tek doğruluk kaynağı sayılıyorsa (kütüphane doktoru bunu varsayıyor) bu beş dosya ya kayda alınmalı ya `bin/`'den ayrılmalı.

> **D5 · `uxclazz` bildirimsiz modül**
>
> `libsx/uxclazz/`, `runtime_ext/uxclz/`, `include/modules/uxclazz.uxmh`, `uxclz.uxmh` ve `uxclz_object.uxmh` birlikte var; manifest yalnız `uxclz`'i tanıyor. Kendi kılavuzu (`UXCLAZZ_UXBDLL_MIMARI_VE_KULLANIM_KILAVUZU_TR.md`) olan bir modülün kütüphane kaydı yok. `uxclazz` mi `uxclz` mi kanonik — karar verilip tek ad kullanılmalı.

Öncelik sırası

Etki × düzeltme maliyeti sırasıyla.

1. **A1 (SQRT).** Tek satırlık karar, en görünür etki. Parser'a `SQRT → SQR` takma adı eklemek beş cildi birden düzeltir; tersi yönde onlarca örnek elle değişecek.
2. **C3 (UNION/PACK/ALIGN layout).** Sessiz yanlış sonuç üreten tek bulgu: derleme başarılı olur, bellek düzeni yanlış olur. C birlikte çalışabilirliğinin temel iddiası burada.
3. **A5 + bayat matrisler.** Üç matris dosyası yanlış yönde iş öneriyor; otomatik üretim kaynağına bağlanmalı veya silinmeli.
4. **D3 (eksik 5 DLL).** Ya derlenmeli ya manifestte `required:false, built:false` gibi açık bir durumla işaretlenmeli; şu an belgeler var sayıyor.
5. **C4/C5 (preprocessor kümeleri).** Hata mesajını düzeltmek ucuz; kanonik listeyi kümelere ayırmak doğru çözüm.
6. **B3 + C6 + D1 (belge üretimi).** Üçü de aynı ilaçla iyileşir: yardım metni, preprocessor tablosu ve fonksiyon atlasları kaynaktan üretilsin. Kayıt zaten makine okunabilir (`cli_registry_adim41.fbs` içinde her seçeneğin `longName/examplePath` alanı var) — `uxb doc` verb'i bu işi yapmak için zaten mevcut.
7. **C1/C2, A2–A4, A6–A10, B1/B2/B4, D2/D4/D5.** Tek tek küçük düzeltmeler.

YÖNTEM VE KAPSAM SINIRI

Bulgular tek tek dosya okumasıyla doğrulandı; hiçbiri derleyici çalıştırılarak sınanmadı (Windows/FreeBASIC 1.10.1 gerekiyor). Kitap tarafında `docs/uxbasic kitap/` (n/g/d üç sürüm), `docs/uxbasic_5_cilt/`, `docs/matrix/` ve kök kılavuzlar tarandı; `.docx` kılavuzlar ve `docs/uxbasic_pck_kitaplar/`, `uxb_pck_book/` setleri bu turda okunmadı — aynı SQRT/INKEY/operatör tabloları oralarda da tekrar ediyorsa bulgu sayısı artar. Kütüphane tarafında `uxmath` örnek olarak tam denetlendi; kalan 44 modülün atlası aynı yöntemle kontrol edilmeli. `_retired/` ve `Snapshoots/` kapsam dışı bırakıldı.

Bu rapor, mevcut `docs/uxbasic kitap/uxbasic_dil_yuzeyi_ve_syntax_analizi.md` §10 denetimini **tekrarlamaz**; oradaki EVENT/THREAD/PARALEL/PIPE eşzamanlılık, native ABI köprüsü ve SHELL TIMEOUT bulguları hâlâ geçerli ve bu listeye eklenmelidir.